

Úloha C (Hammingov kód)

Zadanie GNU Radio – Implementácia Hammingovho kódu (7,4) (Variant B, Úloha C)

Úvod a cieľ zadania Mojou úlohou bolo nahradiť dočasné prepojovacie bloky vo flowgrape Variant_B_Hamming.grc vlastným kóderom a dekóderom pre Hammingov kód (7,4). Cieľom bolo zabezpečiť aby dáta boli proti chybám vznikajúcim v šumovom kanáli a otestovať schopnosť samoopravy chýb.

Cieľom bolo zabezpečiť prenos dát pomocou Hammingovho kódu (7,4). Implementoval som dva bloky typu basic_block:

- **Encoder:** Rozdeľuje vstupný prúd na 4-bitové bloky a dopĺňa ich o 3 paritné bity.
- **Decoder:** Prijíma 7-bitové bloky, počíta syndróm a opravuje prípadnú jednu chybu v bloku.

Pri simulácii v GNU Radiu som sledoval rozdiel medzi nechráneným (Raw) a chráneným (Hamming) prenosom. Pri nastavení mierneho šumu v kanáli dokázal Hammingov dekóder eliminovať všetky chyby (**BER Ham = 0**), zatiaľ čo nechránený prenos už vykazoval chybovosť. Pri extrémnom šume, kde dochádza k viacnásobným chybám v jednom bloku (viac ako 1 bit), sa prejavila teoretická hranica kódu a chybovosť začala mierne stúpať.

Kód pre Encoder:

```
import numpy as np
```

```
from gnuradio import gr
```

```
class blk(gr.basic_block):
```

```
    def __init__(self):
```

```
        gr.basic_block.__init__(self, name='Hamming (7,4) Encoder', in_sig=[np.uint8],  
out_sig=[np.uint8])
```

```
    def general_work(self, input_items, output_items):
```

```
        in0, out0 = input_items[0], output_items[0]
```

```
        n_blocks = min(len(in0) // 4, len(out0) // 7)
```

```
        if n_blocks == 0: return 0
```

```
        in_m = in0[:n_blocks*4].reshape((n_blocks, 4))
```

```
        d1, d2, d3, d4 = in_m[:,0], in_m[:,1], in_m[:,2], in_m[:,3]
```

```
        p1, p2, p3 = d1^d2^d4, d1^d3^d4, d2^d3^d4
```

```
        out_m = np.column_stack((p1, p2, d1, p3, d2, d3, d4))
```

```
        out0[:n_blocks*7] = out_m.flatten()
```

```
        self.consume(0, n_blocks * 4)
```

```
        return n_blocks * 7
```

Kód pre Decoder:

```
import numpy as np
```

```
from gnuradio import gr
```

```
class blk(gr.basic_block):
```

```
    def __init__(self):
```

```
        gr.basic_block.__init__(self, name='Hamming (7,4) Decoder', in_sig=[np.uint8],  
out_sig=[np.uint8])
```

```
    def general_work(self, input_items, output_items):
```

```
        in0, out0 = input_items[0], output_items[0]
```

```
        n_blocks = min(len(in0) // 7, len(out0) // 4)
```

```
        if n_blocks == 0: return 0
```

```
        in_m = in0[:n_blocks*7].copy().reshape((n_blocks, 7))
```

```
        p1, p2, d1, p3, d2, d3, d4 = [in_m[:,i] for i in range(7)]
```

```
        s1, s2, s3 = p1^d1^d2^d4, p2^d1^d3^d4, p3^d2^d3^d4
```

```
        syndrome = s1*1 + s2*2 + s3*4
```

```
        for i in range(n_blocks):
```

```
            if syndrome[i] > 0: in_m[i, syndrome[i]-1] ^= 1
```

```
        out_m = np.column_stack((in_m[:,2], in_m[:,4], in_m[:,5], in_m[:,6]))
```

```
        out0[:n_blocks*4] = out_m.flatten()
```

```
        self.consume(0, n_blocks * 7)
```

```
        return n_blocks * 4
```

Obrázok výstupu:

MPSK channel with Noise and Hamming Encoding

— □ ×

BER MPSK

BER Output without Hamming Encoding BER Output with Hamming Encoding

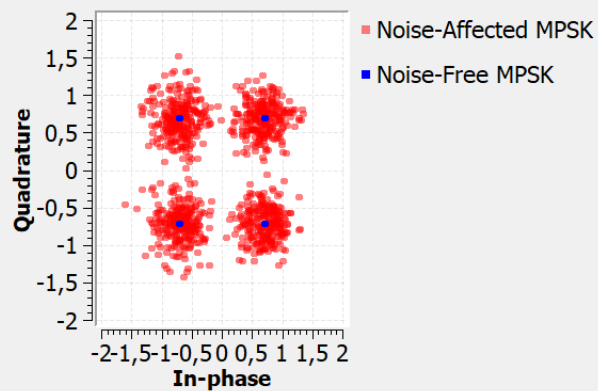
BER

BER

-4.474519 = X

-4.661040 = X

**Comparison of MPSK
Signal
Constellations:
Noise-Free vs. Noise-
Affected**



**MPSK Error Rate
Histogram:
Hamming Code vs.
No ECC**

